BitDegree

RaisedButton

Componente substituído na versão mais nova do Flutter

RaisedButton foi descontinuado e substituído pelo **ElevatedButton**.

Essa mudança ocorreu para alinhar o framework aos padrões mais recentes do Material Design e oferecer uma API de estilização mais flexível através do ButtonStyle.

Além do **RaisedButton**, outros botões clássicos também ganharam sucessores:

- RaisedButton - **ElevatedButton**
- FlatButton - **TextButton**
- OutlineButton - **OutlinedButton**

Flutter

RaisedButton

```
// Antigo (RaisedButton)
RaisedButton(
  child: Text('Enviar'),
  onPressed: () {},
)

// Novo (ElevatedButton)
ElevatedButton(
  child: Text('Enviar'),
  onPressed: () {},
  style: ElevatedButton.styleFrom(
    backgroundColor: Colors.blue, // substitui o antigo 'color'
  ),
)
```

App Perguntas

```
Widget build(BuildContext context){
  final perguntas = [
    'Qual é a sua cor favorita',
    'Qual é o seu animal favorito',
  ];

  return MaterialApp(
    home: Scaffold(
      appBar: AppBar(
        title: Text( 'Perguntas'),
      ), // AppBar
      body: Column(
        children: <Widget>[
          Text (perguntas [0]),
          ElevatedButton(
            child: Text( 'Resposta 1'),
            onPressed: null,
          ), // RaisedButton
        ], // ‹Widget>|]
      ), // Column
    ), // Scaffold
  ); // MaterialApp
}
```

Flutter

App Perguntas

Adicione mais dois botões de resposta

Flutter

App Perguntas

Conectando uma função a um evento

```
class PerguntaApp extends StatelessWidget {
  void responder(){
    print('Pergunta respondida!');
  }
.
.
.
```

Flutter

App Perguntas

Conectando uma função a um evento

```
.
.
.
  return MaterialApp(
    home: Scaffold(
      appBar: AppBar(
        title: Text( 'Perguntas'),
      ), // AppBar
      body: Column(
        children: <Widget>[
          Text (perguntas [0]),
          ElevatedButton(
            child: Text( 'Resposta 1'),
            onPressed: responder,
          ), // RaisedButton
        ], // ‹Widget>|]
      ), // Column
    ), // Scaffold
  ); // MaterialApp
}
```

Flutter

StatelessWidget

O **StatelessWidget** é um componente estático e imutável. Ele funciona como uma "receita" que não muda após ser desenhada na tela.

Pontos principais de como ele opera:

Uma vez que o widget é construído, suas propriedades não podem ser alteradas internamente. Se você precisar mudar algo na tela (como um contador que sobe ou um texto que muda), o StatelessWidget não consegue fazer isso sozinho porque ele não possui o método setState()

Quando usar?

Ele é ideal para partes da interface que dependem apenas de informações enviadas no momento da criação, como:

- Text (Exibições de texto fixo).
- Icon (Ícones decorativos).
- Botões que apenas executam uma ação sem mudar o próprio visual imediatamente.

Flutter

Variáveis em StatelessWidget

```
class PerguntaApp extends StatelessWidget {
  var perguntaSelecionada = 0;

  void responder(){
    print('Pergunta respondida!');
  }
.
.
.
.
.
.
      body: Column(
         children: <Widget>[
           Text (perguntas[perguntaSelecionada]),
.
.
.
```

Com essa alteração, ainda não teremos problemas.
Para testar, altere o valor de perguntaSelecionada e veja que as perguntas mudam.

A questão é que definindo uma variável dentro da classe, estamos adicionando um estado em uma classe que não guarda estado!

Flutter

Variáveis em StatelessWidget

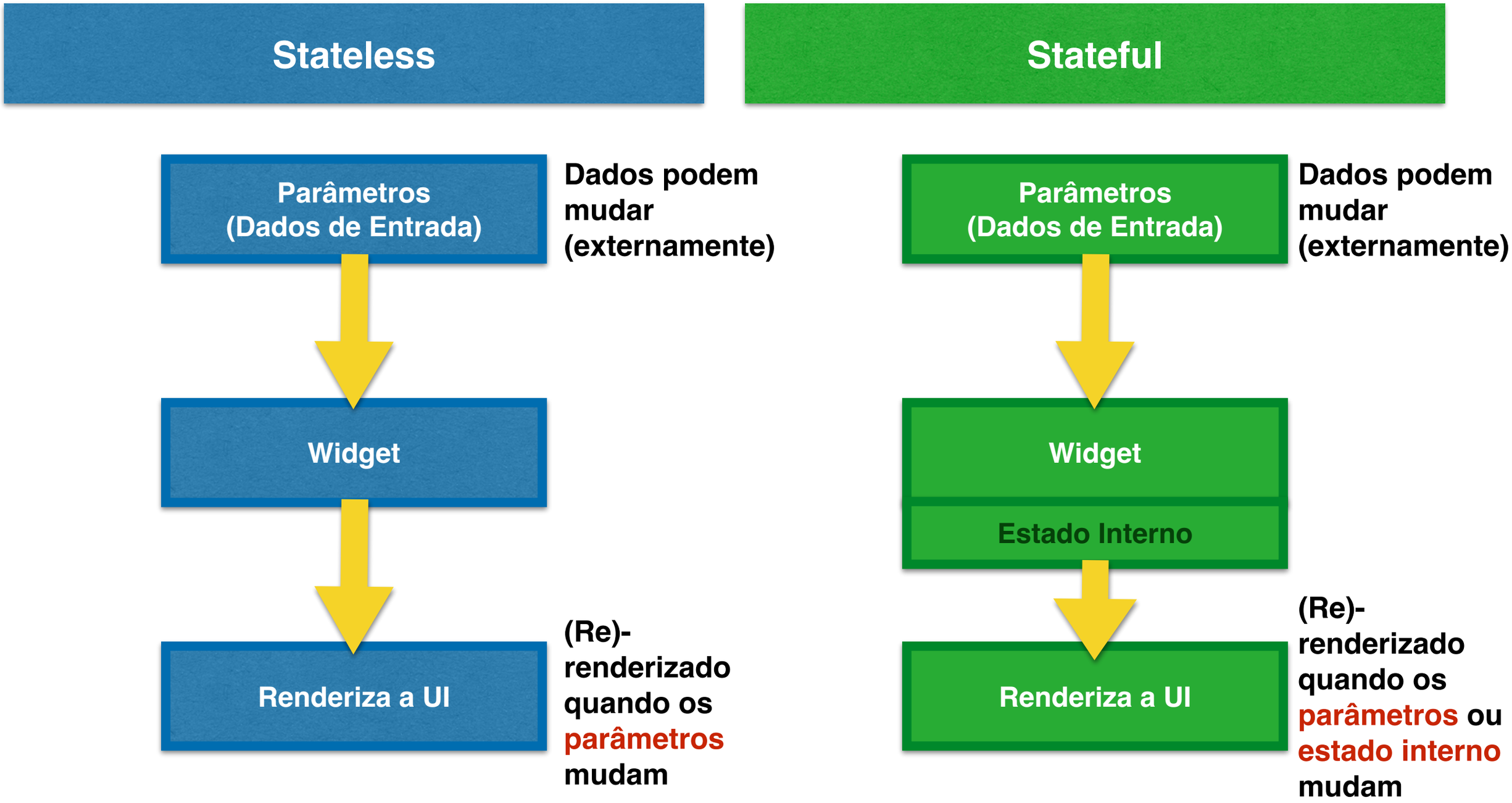
```
class PerguntaApp extends StatelessWidget {
  var perguntaSelecionada = 0;

  void responder(){
    perguntaSelecionada++;
    print(perguntaSelecionada);
  }
.
.
.
.
.
.
     body: Column(
        children: <Widget>[
          Text (perguntas[perguntaSelecionada]),
.
.
.
```

Perceba que o valor vai ser exibido na console, mas as perguntas não serão atualizadas, porque no Flutter as atualizações de estado são refletidas na interface, mas um componente StatelessWidget não guarda estado.

Flutter

Stateless x Stateful



Usando StatefulWidget

Alterando o tipo de widget e criando uma classe para o estado

```
class PerguntaApp extends StatefulWidget {
  var perguntaSelecionada = 0;

  void responder(){
    perguntaSelecionada++;
    print(perguntaSelecionada);
  }
.
.
.

.
.
.
main() => runApp(PerguntaApp());

class PerguntaAppState extends State<PerguntaApp> {
}
.
.
.
```

Usando StatefulWidget

Tudo o que representa estado ou utiliza o estado deve ser colocado na classe PerguntaAppState

- variável
- método responder
- método build

Flutter

Usando StatefulWidget

Implementando o novo método de StatefulWidget

```
class PerguntaApp extends StatefulWidget {
  State<PerguntaApp> createState(){
    return PerguntaAppState();
  }
}


class PerguntaApp extends StatefulWidget {
  var perguntaSelecionada = 0;

  void responder(){
    setState(() {
      perguntaSelecionada++;
    });
    print(perguntaSelecionada);
  }
.
.
.
```